Curso: Tópicos en Aprendizaje Automático

Trabajo Final Integrador: Análisis Crítico de un Artículo Científico

¿Bastan los parches? Análisis y réplica parcial del ConvMixer

Trockman, A. y Kolter, J. Z. (2023). *Patches Are All You Need?* Transactions on Machine Learning Research. arXiv:2201.09792.

Marcos David Kantor

Docentes: Dr. Ing. Martín Morales y Dr. Ing. Marcelo Cappelletti — Universidad Nacional Arturo Jauretche, 2026

Resumen. El artículo de Trockman y Kolter (2023) pregunta si el buen desempeño de los *Vision Transformers* (ViT) se debe a la autoatención o a que la imagen entra dividida en *parches*. Para separar ambos factores propone el ConvMixer, una red que conserva el diseño de parches del ViT pero reemplaza toda la atención por convoluciones estándar, y muestra que iguala o supera a ViT, MLP-Mixer, DeiT, ResMLP y ResNet de tamaño comparable en ImageNet-1k. Analicé el trabajo según la consigna y repliqué parcialmente su metodología en CIFAR-10, en CPU y con un presupuesto de cómputo unas 20 veces menor que el del Apéndice B. En la réplica, un ConvMixer-128/8 alcanza 82,7 % de exactitud de test en 10 épocas, prácticamente igual que una ResNet entrenada con la misma receta (83,0 %), que resulta 3,7 veces más rápida. Se reprodujo el efecto del tamaño de parche, pero no el del tamaño de núcleo: con tan pocas épocas, el núcleo chico rindió más. Con la receta oficial de los autores en una GPU de Colab, el modelo llegó a 94,5 %; con la receta del artículo (200 épocas, mixup y CutMix) en una GPU de Kaggle, a 95,48 %, que iguala el 95,21 % publicado (una sola semilla). Para ubicar el modelo en el programa del curso, lo comparé además con una regresión lineal por seudoinversa (40,7 %), el perceptrón simple (40,3 %), un perceptrón multicapa (57,4 %) y máquinas de aprendizaje extremo (ELM 47,3 %; ML-ELM 40,4 %).

1. Selección del artículo

- **Referencia.** Trockman, A. y Kolter, J. Z. *Patches Are All You Need?* Publicado en *Transactions on Machine Learning Research* (2023); preprint arXiv:2201.09792 (enero de 2022). La versión que se analiza es la enviada a ICLR 2022 (OpenReview), cuyo contenido técnico coincide con la publicada. Código oficial: `https://github.com/locuslab/convmixer`.
- **Cumple la consigna**: es reciente (2022–2023), aborda un problema relevante de AA (clasificación de imágenes y diseño de arquitecturas), presenta un modelo propio, contiene resultados experimentales en dos conjuntos de datos y compara varios algoritmos (ResNet, ViT, DeiT, MLP-Mixer, ResMLP, Swin, MobileNet isotrópica).
- **Ubicación en el programa**: Unidad 4 (CNN): convoluciones 2D, SGD, arquitecturas y modelos de CNN. La réplica toca también las unidades 1, 2, 3 y 5.

2. Introducción

Problema. Desde 2012 las CNN dominaron la visión por computadora. En 2020–2021 el ViT (Dosovitskiy et al., 2021) mostró que un *transformer* puede igualarlas o superarlas si se entrena con suficientes datos. Para aplicar autoatención (cuyo costo crece con el cuadrado del número de elementos) a una imagen, el ViT la corta en parches de 16×16 píxeles y trata cada parche como una "palabra". Ese cambio de representación vino acompañado de un cambio de arquitectura: el ViT es *isotrópico*, es decir, conserva la misma resolución y el mismo ancho en todas sus capas, mientras que las CNN clásicas reducen la resolución y aumentan los canales de forma progresiva. Ambos cambios llegaron juntos, y por eso no se sabe a cuál se debe la mejora. La pregunta del

artículo es: *¿el desempeño del ViT se debe al transformer o, al menos en parte, a usar parches como representación de entrada?*

Importancia para el AA. Es una pregunta sobre el *sesgo inductivo* de los modelos, es decir, qué supuestos sobre los datos están incorporados en la arquitectura. Si gran parte de la ganancia se debe a los parches y al diseño isotrópico, es posible conservar la eficiencia de datos de la convolución (localidad y pesos compartidos) sin pagar el costo cuadrático de la atención. Esto tiene consecuencias prácticas en dominios con pocos datos, como las imágenes médicas. El artículo también discute una práctica común: atribuir el éxito de un modelo nuevo a su operación más llamativa (la atención) sin aislar los demás cambios.

Motivación de los autores. Construir el modelo más simple posible que conserve todo lo que el ViT y el MLP-Mixer tienen en común (parches, isotropía, mezcla espacial separada de la mezcla entre canales) y reemplazar la atención y las MLP por la operación menos expresiva disponible: la convolución. Si un modelo así compite con los originales, hay evidencia de que los parches son parte importante del resultado. Los autores admiten que el título es una exageración.

3. Explicación teórica del modelo

3.1 Tipo de algoritmo y tipo de aprendizaje

El ConvMixer es una red neuronal convolucional profunda para **clasificación** de imágenes, entrenada con **aprendizaje supervisado** (pares imagen–etiqueta, pérdida de entropía cruzada) mediante descenso por gradiente estocástico (AdamW). No usa pre-entrenamiento ni datos externos.

3.2 Funcionamiento general

1. **Embebido de parches.** Una convolución con núcleo $p \times p$ y paso p transforma la imagen $c \times n \times n$ en un tensor $h \times \frac{n}{p} \times \frac{n}{p}$: cada parche se proyecta linealmente a un vector de dimensión h. A diferencia del ViT, los parches conservan su disposición espacial ("tensor layout").

$$z_0 = \mathrm{BN}(\sigma\{\mathrm{Conv}_{c\to h}(X,\ \text{stride} = p,\ \text{kernel} = p)\}) \tag{1}$$

2. d **bloques ConvMixer**, todos iguales y sin submuestreo (isotropía):

$$z'_l = \mathrm{BN}(\sigma\{\mathrm{ConvDepthwise}_{k\times k}(z_{l-1})\}) + z_{l-1} \qquad \text{(mezcla espacial, con residual)} \tag{2}$$

$$z_{l+1} = \mathrm{BN}(\sigma\{\mathrm{ConvPointwise}_{1\times 1}(z'_l)\}) \qquad \text{(mezcla entre canales)} \tag{3}$$

   La convolución *depthwise* aplica un filtro $k \times k$ distinto a cada canal (no mezcla canales); la *pointwise* (1×1) combina los h canales en cada posición. Juntas forman una convolución separable, como en MobileNet. La activación σ es GELU y la normalización es BatchNorm.

3. **Clasificador**: *pooling* promedio global, que produce un vector de dimensión h, y una capa lineal con *softmax*.

El modelo completo se escribe en unas seis líneas de PyTorch (Fig. 3 del artículo). Mi implementación (`src/modelos.py`) transcribe su Fig. 7.

3.3 Principales características y parámetros

- **Cuatro hiperparámetros de arquitectura**: ancho h, profundidad d, tamaño de parche p y tamaño de núcleo k. Se nombra **ConvMixer-h/d**. El número de parámetros tiene la forma cerrada (ec. 4): $\#\theta = h\,[\,d(k^2 + h + 6) + c\,p^2 + n_{\text{clases}} + 3\,] + n_{\text{clases}}$; el término dominante es $d\,h^2$, que corresponde a las convoluciones 1×1.

- **Núcleos inusualmente grandes** ($k = 7$ a 9, frente al $k = 3$ habitual), para mezclar posiciones lejanas como lo hacen la atención o las MLP de mezcla espacial. El campo receptivo crece como $p\,[1 + d(k-1)]$ píxeles.
- **Resolución interna** n/p constante. Con parches chicos es alta, lo que da buena exactitud a costa de mucho cómputo y memoria.

3.4 Modelos comparados que no se ven en el curso

ResNet
(He et al., 2016) CNN piramidal con conexiones residuales $y = F(x) + x$, que permiten entrenar redes de más de cien capas. Es la línea de base clásica.

ViT / DeiT
(Dosovitskiy et al., 2021; Touvron, Cord et al., 2021) *Transformer* sobre parches de 16×16: cada capa alterna autoatención multi-cabeza (cada parche pondera a todos los demás según su similitud, $\mathrm{softmax}(QK^\top/\sqrt{d})V$) y una MLP por parche. DeiT es un ViT entrenado con más regularización para que funcione con ImageNet-1k solo.

MLP-Mixer / ResMLP
(Tolstikhin et al., 2021; Touvron, Bojanowski et al., 2021) Reemplazan la atención por una MLP que actúa a lo largo de los parches (mezcla espacial) y otra a lo largo de los canales. El ConvMixer toma de ellos la idea de *mezclar* y la implementa con convoluciones.

Swin
(Liu et al., 2021) Transformer jerárquico con atención en ventanas locales desplazadas. Es más complejo y se incluye como referencia del estado del arte.

3.5 Ventajas, limitaciones y rasgos distintivos

Ventajas: simplicidad extrema; buena exactitud por parámetro; sesgo inductivo convolucional (localidad, pesos compartidos), que da eficiencia de datos (más de 96 % en CIFAR-10 con 0,7 M parámetros); acepta entradas de tamaño variable; su alta resolución interna lo hace candidato natural para segmentación. **Limitaciones**: baja velocidad de inferencia (*throughput*) por los parches chicos y los núcleos depthwise grandes, que las librerías no optimizan bien; mucha memoria; no es el estado del arte (Swin lo supera entre 1,2 y 1,6 puntos y es entre 4 y 6 veces más rápido).

4. Metodología del artículo

Enfoque. Es un estudio empírico con dos partes. La primera es una comparación en ImageNet-1k contra modelos isotrópicos basados en parches (DeiT, ResMLP, ViT, MLP-Mixer) y contra ResNet entrenadas con la misma receta. La segunda es un conjunto de ablaciones y de barridos de diseño en CIFAR-10 (Apéndice B).

Datos.

- *ImageNet-1k*: unos 1,28 M de imágenes de entrenamiento y 50 000 de validación, 1 000 clases, entrada de 224×224. Sin pre-entrenamiento ni datos adicionales.
- *CIFAR-10*: 50 000 imágenes de entrenamiento y 10 000 de prueba, de 32×32 en color, 10 clases balanceadas (avión, auto, pájaro, gato, ciervo, perro, rana, caballo, barco, camión).

Preprocesamiento y selección de características. No hay selección explícita de variables: la capa de parches *aprende* los rasgos (filtros tipo Gabor y detectores de color, Apéndice C). El preprocesamiento consiste en normalizar por canal y aplicar una aumentación de datos intensa con

la biblioteca `timm`: RandAugment, mixup, CutMix, *random erasing*, escalado aleatorio y recorte de la norma del gradiente. La ablación de la Tabla 3 muestra que esta aumentación es decisiva en CIFAR-10: sin escalado aleatorio la exactitud cae de 95,88 % a 86,24 %; sin recorte de gradiente, a 86,33 %; sin RandAugment, a 92,86 %.

Partición, entrenamiento y validación. En ImageNet se usa la partición estándar y se informa la exactitud top-1 en el conjunto de validación oficial. En CIFAR-10 se usan los 50 000 de entrenamiento y se informa sobre los 10 000 de prueba. Se entrena con AdamW (Loshchilov y Hutter, 2019) y un programa de tasa de aprendizaje triangular: entre 90 y 300 épocas en ImageNet (unos 9 días en 10 GPU RTX8000 para el modelo mayor) y 200 épocas en CIFAR-10.

Optimización de hiperparámetros. Los autores dicen que *no hicieron ninguna* búsqueda en ImageNet: fijaron parámetros "de sentido común" que funcionaron en un ConvMixer-1024/12 y los mantuvieron para todos los modelos. En CIFAR-10 sí ajustaron h, d, p, k y el *weight decay* (Tabla 4) a mano, sin búsqueda sistemática y, por lo que se informa, sin un conjunto de validación separado del de prueba.

Métricas. Exactitud top-1, número de parámetros y *throughput* (imágenes por segundo en una RTX8000, lotes de 64, fp16).

5. Resultados del artículo

Tabla 1

Resultados del artículo en ImageNet-1k (selección de sus Tablas 1 y 2)

Modelo	Parche	Parámetros (M)	Img/s	Épocas	Top-1 ImageNet (%)
ConvMixer-1536/20	7	51,6	89	150	**81,37**
ConvMixer-768/32	7	21,1	203	300	**80,16**
ResNet-152 (misma receta)	–	60,2	872	150	79,64
DeiT-B	16	86	703	300	81,8
ResMLP-B24/8	8	129	140	400	81,0
ViT-B/16	16	86	704	300	77,9
Mixer-B/16	16	59	816	300	76,44
Swin-T	4	29	884	300	81,3

Nota. Imágenes de 224×224, sin datos adicionales. Img/s medido por los autores en una GPU RTX8000 (lote 64, fp16). Sólo los ConvMixer y la ResNet se entrenaron con la misma receta; el resto de los valores proviene de los artículos originales. Adaptado de Trockman y Kolter (2023, Tablas 1 y 2).

- **Principal**: con un presupuesto de parámetros similar, el ConvMixer supera a ResNet, ViT, MLP-Mixer y ResMLP, y queda cerca de DeiT. El ConvMixer-768/32 iguala a la ResNet-152 con un tercio de sus parámetros.
- **Comparación con modelos tradicionales**: la ResNet entrenada con exactamente la misma receta queda por debajo en exactitud, pero es mucho más rápida (872 frente a 89 imágenes por segundo).
- **Ablaciones**: los núcleos grandes ayudan (ImageNet: $-0{,}94$ puntos al pasar de $k = 9$ a $k = 3$; CIFAR-10: 93,61 % con $k = 3$ frente a 95,88 % con $k = 9$); los parches más chicos rinden más (78,9 % con $p = 14$ frente a 81,4 % con $p = 7$; en CIFAR-10, 95,0 % con $p = 2$ frente a 92,6 % con $p = 4$ y 85,6 % con $p = 8$). Cambiar GELU por ReLU o BatchNorm por LayerNorm afecta poco.
- **CIFAR-10**: más de 91 % con 0,1 M parámetros y más de 96 % con 0,9 M.

6. Discusión y conclusiones del artículo

Conclusiones de los autores. El diseño isotrópico con embebido de parches es por sí mismo una plantilla potente: los parches hacen todo el submuestreo de una sola vez, lo que agranda el campo receptivo efectivo y facilita mezclar información espacial lejana. La atención no es lo único que la visión tomó del procesamiento de lenguaje; *tokenizar* la entrada también lo es.

Limitaciones, algunas reconocidas por los autores y otras que encontré al leerlo:

1. *La pregunta no queda respondida del todo.* Para separar el efecto de los parches haría falta un experimento factorial: el mismo modelo con y sin parches, y modelos piramidales con y sin atención. El artículo compara arquitecturas que difieren en varios factores a la vez. La línea de base más cercana, la MobileNet isotrópica de Sandler et al. (2019), ya había mostrado algo similar antes que el ViT.
2. *Comparaciones con recetas distintas.* Los números de DeiT, ResMLP, ViT y Swin se copian de sus artículos, con otras recetas y otra cantidad de épocas. Sólo las ResNet se entrenaron en igualdad de condiciones.
3. *Una sola corrida por configuración*, sin intervalos de confianza. Diferencias de 0,2 a 0,4 puntos, como las que separan al ConvMixer de DeiT, pueden estar dentro de la variabilidad entre semillas.
4. *Hiperparámetros de CIFAR-10 elegidos mirando el conjunto de prueba*, sin validación separada, lo que introduce un sesgo optimista (Unidad 1: el test se usa una sola vez).
5. *Costo*: el *throughput* es entre 4 y 10 veces menor que el de los competidores de tamaño similar, y el modelo mayor necesitó unos 90 días-GPU de cómputo.
6. *Variables de confusión*: GELU frente a ReLU, cantidad de épocas y tamaño de parche distinto del de DeiT. Los autores atenúan las dos primeras con un experimento con ReLU.

Trabajo futuro propuesto. ConvMixers más profundos con parches más grandes y más regularización; optimización de bajo nivel de la convolución depthwise de núcleo grande; cuellos de botella o un clasificador más expresivo; segmentación semántica y detección; pre-entrenamiento a gran escala; experimentos que aíslen mejor el efecto de los parches. Trabajos posteriores, como ConvNeXt (Liu et al., 2022) y RepLKNet (Ding et al., 2022), confirmaron que los núcleos grandes y las recetas modernas cierran la brecha entre CNN y transformers.

7. Réplica parcial de la metodología

7.1 Diseño

Repliqué el **experimento de CIFAR-10 del Apéndice B** con el mismo conjunto de datos, que es público (Krizhevsky, 2009; descarga oficial de la Universidad de Toronto: `https://www.cs.toronto.edu/~kriz/cifar-10.html`, archivo `cifar-10-python.tar.gz`, 170 MB), y mantuve fijos todos los elementos de la metodología que el hardware disponible permitía. El código, los hiperparámetros y las semillas están en el repositorio (`src/`, `config.json`) y en el notebook `notebooks/replica_convmixer.ipynb`.

- **Igual que el artículo**: arquitectura (transcripción literal de la Fig. 7), conjunto de datos, AdamW, programa triangular de tasa de aprendizaje (el del repositorio oficial), recorte de la norma del gradiente (1,0), escalado aleatorio, recorte con relleno y espejo, *weight decay* 10^{-2}, métrica (exactitud) y conteo de parámetros por la ec. (4), que verifiqué contra el conteo de PyTorch.

- **Distinto, por cómputo**: usé la CPU de mi computadora (8 núcleos, sin GPU), 10 épocas en lugar de 200 y el modelo ConvMixer-128/8 con $p = 2$ en lugar de 256/8 con $p = 1$, que en este equipo tardaría unas 3,5 horas *por época*. No usé RandAugment, mixup, CutMix ni *random erasing*, porque con pocas épocas la regularización fuerte retrasa la convergencia.
- **Mejora metodológica**: separé 5 000 imágenes de entrenamiento como validación. Evalué el test de 10 000 una sola vez, al final. Los modelos clásicos eligieron su regularización por validación cruzada de 5 pliegues (seudoinversa) o por el conjunto de validación (ELM).
- **Comparaciones agregadas**: la ResNet con la misma receta (como en el artículo), ablaciones de núcleo ($k = 3$) y de parche ($p = 4$), y cinco modelos del programa del curso sobre píxeles crudos: asociador lineal resuelto por seudoinversa con regularización de Tikhonov, perceptrón promediado (Freund y Schapire, 1999), MLP, ELM (Huang et al., 2006) con su versión regularizada (Huang et al., 2012) y ML-ELM (Kasun et al., 2013).

7.2 Resultados

Tabla 2

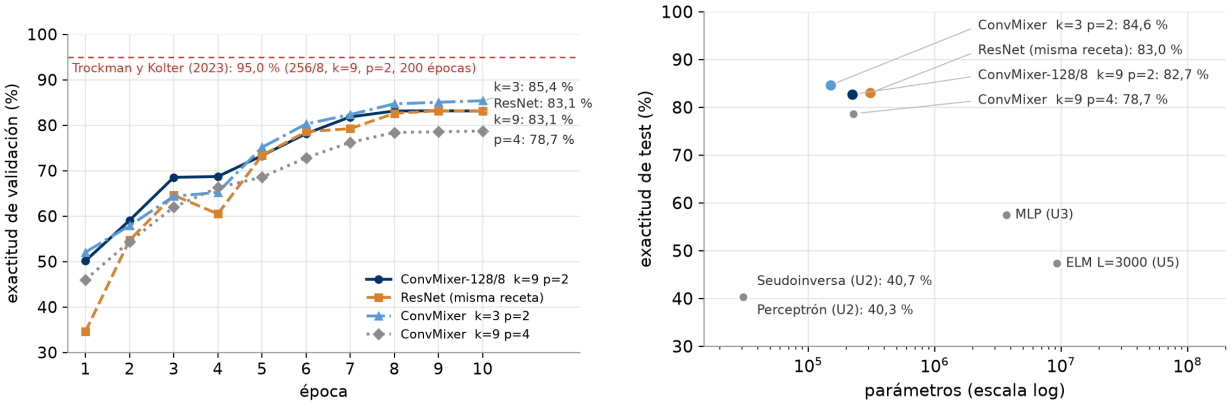
Réplica en CIFAR-10: exactitud de test de los modelos de las cinco unidades

Unidad	Modelo	Parámetros	Test (%)	Img/s (CPU)	Entrenamiento
U4	**ConvMixer-128/8**, $k = 9$, $p = 2$	223 370	**82,7**	422	110 min
U4	ConvMixer-128/8, $k = 3$, $p = 2$	149 642	84,6	531	72 min
U4	ConvMixer-128/8, $k = 9$, $p = 4$	227 978	78,7	1551	35 min
U4	ResNet (32-64-128), misma receta	308 650	83,0	1572	32 min
U3	MLP 3072-1024-512-10 (AdamW)	3 676 682	57,4	14147	–
U5	ELM ($L = 3000$, λ por validación)	9 249 000	47,3	–	–
U5	ML-ELM (1000-1000 + ELM 3000)	–	40,4	–	–
U2	Seudoinversa ($\lambda = 10000$ por CV)	30 730	40,7	–	–
U2	Perceptrón simple multiclase (promediado)	30 730	40,3	–	–
	Artículo: ConvMixer-256/8, $k = 9$, $p = 2$, 200 ép., GPU	709 k	95,00	–	–

Nota. Test de 10 000 imágenes, evaluado una sola vez. Las CNN usan una semilla; MLP, ELM, ML-ELM y perceptrón, el promedio de 3 semillas. Img/s: inferencia en la misma CPU (8 núcleos) con lotes de 64. Entrenamiento: minutos por corrida, sólo para las CNN, que corrieron solas en la máquina; los modelos clásicos compartieron la CPU con otro proceso y sus tiempos no son comparables. La última fila es el valor publicado (Trockman y Kolter, 2023, Tabla 4).

Figura 1

Curvas de validación de las CNN y exactitud frente a parámetros de todos los modelos



Nota. Izquierda: exactitud en las 5 000 imágenes de validación por época; la línea roja es el valor publicado para ConvMixer-256/8 con 200 épocas. Derecha: exactitud de test frente al número de parámetros (escala logarítmica). Elaboración propia a partir de `results/*.json`.

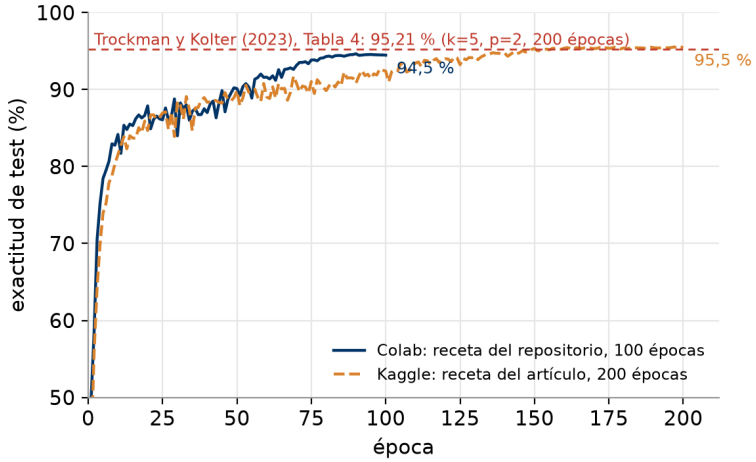
7.3 Comparación con el artículo y causas de las diferencias

Valor absoluto. El ConvMixer-128/8 llegó a 82,7 % en test, frente a 95,0 % que informa el artículo para la misma configuración de núcleo y parche ($k = 9$, $p = 2$) con ancho 256. La diferencia de unos 12 puntos tiene causas concretas:

1. *Entrenamiento 20 veces más corto* (10 épocas frente a 200). La Figura 1 muestra que la validación se estabiliza en las últimas épocas sólo porque la tasa de aprendizaje llega a cero, no porque el modelo haya terminado de aprender.
2. *Modelo 3 veces más chico* (223 k frente a 709 k parámetros). En la Tabla 4 del artículo, pasar de $h = 256$ a $h = 128$ cuesta entre 1 y 2 puntos aun con 200 épocas.
3. *Sin RandAugment, mixup ni CutMix.* Según la Tabla 3 del artículo, quitar sólo RandAugment cuesta 3 puntos.
4. *Protocolo más estricto*: el test se miró una sola vez y los hiperparámetros no se ajustaron sobre él.

Para comprobar que la brecha se debe al cómputo y no a un error de implementación, entrené además el modelo en una GPU gratuita de Google Colab (Tesla T4) con la receta del repositorio oficial de los autores para CIFAR-10 (`https://github.com/locuslab/convmixer-cifar10`): ConvMixer-256/8 con $k = 5$ y $p = 2$, RandAugment, ColorJitter, *random erasing* y 100 épocas, a unos 56 segundos por época. Esa corrida llegó a 94,5 % en test al cabo de las 100 épocas (Figura 2). Una primera corrida idéntica, que se cortó en la época 84 al perderse la sesión gratuita, ya había llegado a 94,50 % (registro en `results/colab_gpu_corrida1_log.txt`): las dos corridas coinciden. El artículo informa 95,21 % para esa configuración con 200 épocas. Es decir: con la receta del repositorio la implementación queda a menos de un punto del valor publicado. Para cerrar esa brecha corrí después, en una GPU de Kaggle (Colab ya no nos daba GPU), la receta del artículo: 200 épocas, mixup, CutMix y sin *weight decay*. Llegó a 95,48 % en la última época (Figura 2), frente al 95,21 % publicado. La diferencia de la réplica en CPU se explica, entonces, por la escala del experimento.

Figura 2
Réplica en GPU: receta del repositorio (Colab, 100 épocas) y receta del artículo (Kaggle, 200 épocas)



Nota. Exactitud de test por época de ConvMixer-256/8 ($k = 5$, $p = 2$), sólo para seguimiento: el valor que se informa es el de la última época, sin elegir la mejor. La línea roja es el valor del artículo para la misma configuración con 200 épocas. Elaboración propia con el script `src/colab_receta_paper.py`.

La Tabla 3 resume la comparación que me parece más interesante para discutir. Hay tres lecturas. Primero, en un tiempo de reloj parecido (entre 1,5 y 2 horas), la GPU entrenó un modelo 2,7

Tabla 3
La misma metodología con distinto cómputo: CPU propia, GPU de Colab, GPU de Kaggle y artículo

	CPU propia (8 núcleos)	Google Colab (T4)	Kaggle (T4)	Artículo (GPU)
Modelo	128/8, $k = 9$, $p = 2$	256/8, $k = 5$, $p = 2$	256/8, $k = 5$, $p = 2$	256/8, $k = 9$ o 5, $p = 2$
Parámetros	223 370	594 186	594 186	709 k / 594 k
Aumentación	escalado, recorte y espejo	+ RandAugment, ColorJitter, borrado	Colab + mixup y CutMix	`timm` + mixup y CutMix
Weight decay	0,01	0,005	0	0 (Tabla 4)
Épocas	10	100	200	200
Tiempo por época	11 min	56 s	25 s	no informado
Tiempo total	110 min	93 min	85 min	no informado
Exactitud de test	82,7 %	94,5 %	95,48 %	95,0 % / 95,21 %

Nota. Una corrida por configuración, sin repetir semillas. Colab y Kaggle: se informa la última época, sin elegir la mejor; ambas con una GPU Tesla T4 (en Kaggle la sesión tenía dos, pero el script usa una). Colab entrenó al cabo de las 100 épocas. Valores del artículo: Trockman y Kolter (2023, Tabla 4). Antes de la corrida de Colab que se informa hubo otra idéntica que se cortó en la época 84 (94,50 %); el registro está en `results/`.

veces más grande, con muchas más épocas y con la aumentación completa, y ganó unos 12 puntos con el mismo código. Esto relativiza las comparaciones entre arquitecturas: una parte de lo que se atribuye a "un modelo mejor" corresponde a "un modelo que recibió más cómputo y una receta más trabajada". Es justamente la crítica que se le puede hacer al artículo cuando compara contra números de otros trabajos.

Segundo, la receta cuenta. Pasar de la receta del repositorio (100 épocas, 94,5 %) a la del artículo (200 épocas, mixup, CutMix y sin *weight decay*) subió la exactitud de 94,5 % a 95,48 %, es decir, un punto, y quedó 0,27 puntos por encima del 95,21 % publicado. Con una sola semilla por configuración, esa diferencia con el artículo está dentro de lo que cabe esperar por azar (del orden de 0,2 a 0,3 puntos), así que lo prudente es decir que la réplica *iguala* el resultado publicado y no que lo supera. Tampoco puedo separar cuánto aportó cada cambio (las épocas, mixup y CutMix, o el *weight decay*), porque cambié todo junto; sólo el artículo muestra que quitar mixup y CutMix casi no cambia el resultado (Tabla 3), lo que sugiere que el grueso viene de las épocas.

Tercero, el costo. Con la misma GPU, la corrida de Kaggle tardó unos 85 minutos (25 s por época) y la de Colab unos 93 minutos con la mitad de las épocas (56 s por época): en Kaggle cada época fue más del doble de rápida. No sé a qué atribuirlo con certeza: la GPU es la misma (Tesla T4), pero cambió la cantidad de procesos que cargan y aumentan las imágenes (4 en Kaggle y 2 en Colab) y, con RandAugment, esa parte corre en la CPU y puede ser el cuello de botella. En la práctica, Colab gratuito nos negó la GPU por límite de uso a la mañana siguiente y había cortado una corrida en la época 84; Kaggle ofrece unas 30 horas semanales y permite dejar la corrida en segundo plano sin tener la pestaña abierta, lo que la hizo más cómoda y confiable para un entrenamiento largo. **Ablaciones: qué se reprodujo y qué no.**

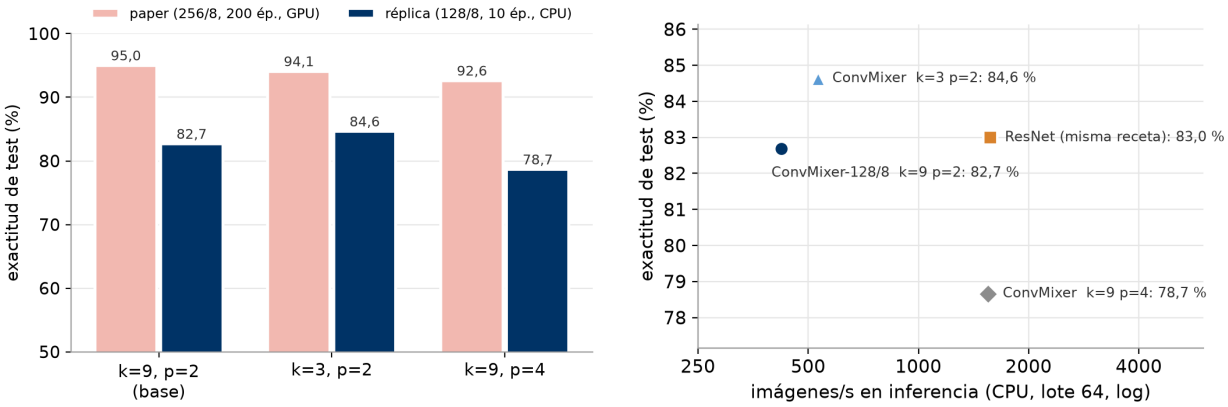
- *Tamaño de parche: se reprodujo.* Con $p = 4$ la exactitud cae a 78,7 % (−4,0 puntos frente a $p = 2$; en el artículo, −2,4). Con parches más grandes la resolución interna baja de 16×16 a 8×8 y el modelo pierde detalle, aunque corre unas 3,7 veces más rápido.
- *Tamaño de núcleo: no se reprodujo.* Con $k = 3$ obtuve 84,6 %, es decir, **2,0 puntos a favor del núcleo chico**, al revés que en el artículo (+2,3 a favor de $k = 9$ con 200 épocas). Mi interpretación es que, con sólo 10 épocas, el modelo con menos parámetros y un sesgo local más fuerte converge antes, mientras que los núcleos grandes necesitan más entrenamiento para aprovechar el campo receptivo amplio. Los propios autores admiten esta posibilidad ("*It could be that with more training iterations or more regularization, the effect of larger kernels would be more pronounced*", Apéndice B). Con una sola semilla por modelo, una diferencia de 2 puntos es indicativa pero no concluyente.
- *ConvMixer frente a ResNet: empate.* La ResNet entrenada con la misma receta obtuvo 83,0 % frente a 82,7 % del ConvMixer ($k = 9$), una diferencia de 0,3 puntos, menor que la variabilidad esperable entre semillas. El artículo encuentra una ventaja clara del ConvMixer en ImageNet,

con modelos 100 veces más grandes y 150 épocas. En esta escala no la observo.

- *Velocidad: se reprodujo.* En inferencia, la ResNet procesa 1572 imágenes por segundo y el ConvMixer con $k = 9$ y $p = 2$, 422: la ResNet es 3,7 veces más rápida, en línea con las Tablas 1 y 2 del artículo. El entrenamiento muestra lo mismo: 110 minutos el ConvMixer y 32 minutos la ResNet.

Figura 3

Ablaciones de la réplica frente a la Tabla 4 del artículo, y velocidad frente a exactitud



Nota. Izquierda: exactitud de test de la réplica (ConvMixer-128/8, 10 épocas, CPU) y del artículo (ConvMixer-256/8, 200 épocas, GPU; Trockman y Kolter, 2023, Tabla 4). Derecha: imágenes por segundo en inferencia, medidas en la misma CPU con lotes de 64 (eje logarítmico). Elaboración propia.

Los modelos del programa. El orden de exactitud reproduce el recorrido del curso: perceptrón (40,3 %) ≈ seudoinversa (40,7 %) < ELM (47,3 %) < MLP (57,4 %) ≪ CNN (83,0 %–84,6 %). Los dos modelos lineales empatan porque comparten la misma limitación: una frontera lineal sobre los 3 072 píxeles. La validación cruzada eligió una regularización fuerte ($\lambda = 10000$), señal de que el sistema está mal condicionado (píxeles vecinos casi colineales). La ELM mejora al agregar no linealidad, pero sus rasgos aleatorios no aprovechan la estructura espacial; además, con $N = 45\,000 \gg L = 3\,000$ casi no sobreajusta, y su exactitud seguía subiendo con L, de modo que el límite fue la memoria y no el método. El ML-ELM (40,4 %) quedó por debajo de la ELM simple: sus autoencoders comprimen los píxeles a 1 000 dimensiones con una reconstrucción lineal y pierden información útil para clasificar. El MLP, con 16 veces más parámetros que el ConvMixer, queda 25 puntos abajo y sobreajusta (76 % en entrenamiento frente a 58 % en validación). Es la mejor ilustración del *sesgo inductivo*: la localidad y los pesos compartidos de la convolución valen más que los parámetros.

Limitaciones de esta réplica. Una sola semilla por CNN, por costo de cómputo; 10 épocas; ningún ajuste de hiperparámetros para la ResNet, igual que en el artículo; y modelos clásicos sobre píxeles crudos, sin extracción de rasgos. Las diferencias menores a un punto no deben interpretarse.

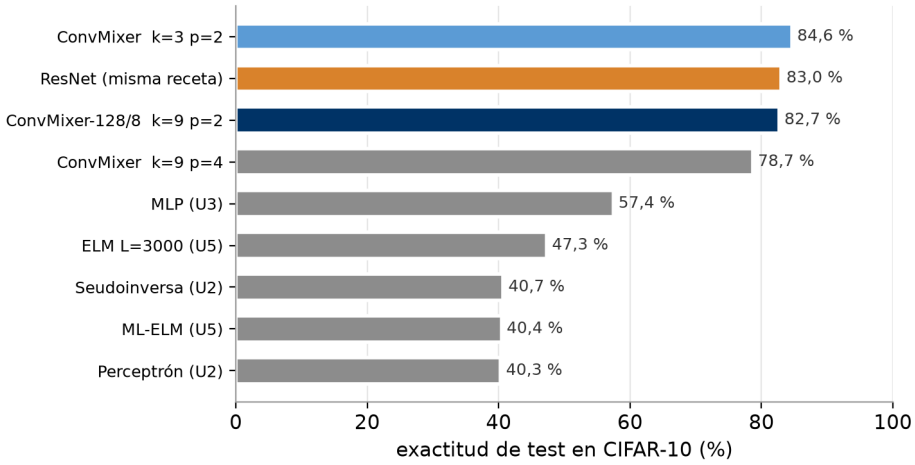
8. Implicancias y reflexión crítica

¿Por qué este artículo? Por tres razones. Primero, forma parte de la bibliografía que trabajé en la materia *Sistemas de Inteligencia Artificial* del ITBA (clase de Transformers y ViT), y analizarlo con la lente de la Unidad 4 permite unir ambos cursos. Segundo, tiene una pregunta científica clara que admite una réplica honesta en hardware modesto, porque el modelo cabe en seis líneas. Tercero, se vincula con mi tesis doctoral (evaluación de la inestabilidad espinal con IA sobre tomografías y radiografías), donde la elección entre CNN, transformers y modelos híbridos es una decisión de diseño concreta y los conjuntos de datos son chicos.

¿En qué otros dominios podría usarse? En *segmentación de imágenes médicas*, porque la

Figura 4

Exactitud de test de todos los modelos de la réplica en CIFAR-10



Nota. Azul oscuro: modelo del artículo; naranja: línea de base ResNet; gris: ablaciones y modelos de las unidades 2, 3 y 5. Promedio de 3 semillas en los modelos no convolucionales. Elaboración propia.

resolución interna constante evita la pérdida de detalle del submuestreo y los núcleos grandes capturan contexto anatómico amplio (por ejemplo, para la segmentación vertebral). También en microscopía y teledetección (imágenes grandes con texturas repetitivas), en señales 1D como ECG o vibraciones (un ConvMixer 1D es inmediato) y en espectrogramas de audio. Su eficiencia de datos lo hace atractivo cuando no hay millones de ejemplos.

Conocimientos previos necesarios. Convolución 2D y campo receptivo, convoluciones depthwise y pointwise (MobileNet), BatchNorm, conexiones residuales, GELU, AdamW y programas de tasa de aprendizaje, técnicas de aumentación (RandAugment, mixup, CutMix) y, para entender la motivación, el mecanismo de autoatención y el ViT. De este curso fueron necesarias las unidades 3 (retropropagación y optimización) y 4 (convolución, SGD y arquitecturas); la Unidad 1 fue clave para criticar la metodología (validación y test).

Implicancias éticas.

- *Costo ambiental y desigualdad de acceso*: 90 días-GPU para un punto de exactitud en ImageNet. Esta réplica muestra el otro lado: sin GPU sólo se puede verificar una versión reducida, y la investigación queda concentrada en quien tiene cómputo. Los autores lo reconocen ("*limited compute*").
- *Reproducibilidad*: publicar el código y una implementación mínima es una buena práctica. Informar una sola corrida sin incertidumbre y ajustar mirando el test va en sentido contrario.
- *Sesgos de los datos*: ImageNet tiene sesgos geográficos y culturales documentados, y un modelo "mejor" en ImageNet no es necesariamente mejor para otra población. En aplicaciones médicas eso exige validar externamente en datos locales.
- *Doble uso*: los clasificadores de imágenes eficientes sirven tanto para diagnóstico como para vigilancia masiva.

Ventajas y limitaciones frente a otros algoritmos. La réplica ordena con claridad los algoritmos del programa. Los modelos lineales (U2) no superan el 40,7 % porque una frontera lineal sobre píxeles no captura la estructura de las clases. La ELM (U5) mejora con más neuronas aleatorias y se entrena en minutos sin gradiente, pero sus rasgos aleatorios no aprovechan la geometría de la imagen. El MLP (U3) tiene 16 veces más parámetros que el ConvMixer y queda muy por debajo. Las CNN (U4) dominan porque incorporan localidad y pesos compartidos. Entre ellas, el ConvMixer logra la misma exactitud que la ResNet con menos parámetros, pero es bastante más lento;

esa parte de la conclusión del artículo se mantiene en mi réplica, no así su ventaja en exactitud.

Uso de asistentes de IA

Usé un asistente de IA (Claude Code) como apoyo para la programación y la revisión de la redacción. La verificación de los resultados fue mía: comparé el conteo de parámetros con la ec. (4), evalué el test una sola vez, generé todos los números citados desde `results/*.json` con `src/figuras.py` y contrasté las afirmaciones sobre el artículo con su texto.

Referencias

Ding, X., Zhang, X., Han, J. y Ding, G. (2022). Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 11963–11975). https://doi.org/10.48550/arXiv.2203.06717

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. y Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. En *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2010.11929

Freund, Y. y Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning, 37*(3), 277–296.

He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep residual learning for image recognition. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Huang, G.-B., Zhou, H., Ding, X. y Zhang, R. (2012). Extreme learning machine for regression and multiclass classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part B, 42*(2), 513–529.

Huang, G.-B., Zhu, Q.-Y. y Siew, C.-K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing, 70*(1–3), 489–501. https://doi.org/10.1016/j.neucom.2005.12.126

Kasun, L. L. C., Zhou, H., Huang, G.-B. y Vong, C. M. (2013). Representational learning with ELMs for big data. *IEEE Intelligent Systems, 28*(6), 31–34.

Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images* [Informe técnico]. University of Toronto. https://www.cs.toronto.edu/~kriz/cifar-10.html

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S. y Guo, B. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. En *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 10012–10022). https://doi.org/10.48550/arXiv.2103.14030

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T. y Xie, S. (2022). A ConvNet for the 2020s. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 11976–11986). https://doi.org/10.48550/arXiv.2201.03545

Loshchilov, I. y Hutter, F. (2019). Decoupled weight decay regularization. En *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.1711.05101

Sandler, M., Baccash, J., Zhmoginov, A. y Howard, A. (2019). Non-discriminative data or weak model? On the relative importance of data and model resolution. En *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*.

Tolstikhin, I., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., Lucic, M. y Dosovitskiy, A. (2021). MLP-Mixer: An all-MLP architecture for vision. *Advances in Neural Information Processing Systems, 34*, 24261–24272. https://doi.org/10.48550/arXiv.2105.01601

Touvron, H., Bojanowski, P., Caron, M., Cord, M., El-Nouby, A., Grave, E., Joulin, A., Synnaeve, G., Verbeek, J. y Jégou, H. (2021). *ResMLP: Feedforward networks for image classification with data-efficient training* [Preimpresión]. arXiv. https://doi.org/10.48550/arXiv.2105.03404

Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A. y Jégou, H. (2021). Training data-efficient image transformers & distillation through attention. En *Proceedings of the 38th International Conference on Machine Learning* (pp. 10347–10357). PMLR. https://doi.org/10.48550/arXiv.2012.12877

Trockman, A. y Kolter, J. Z. (2023). Patches are all you need? *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2201.09792